ESCOLA POLITÉCNICA DA UNIVERSIDADE DE SÃO PAULO

DEPARTAMENTO DE ENGENHARIA DE COMPUTAÇÃO E SISTEMAS DIGITAIS



RELATÓRIO DE SISTEMAS DIGITAIS

Simulação do Processador LEGv8

Nome do Aluno – NUSP XXXXXXX
Nome do Aluno – NUSP XXXXXXX
Nome do Aluno – NUSP XXXXXXX
Nome do Aluno – NUSP XXXXXXX

8 de outubro de 2026

Sumário

1 Seção com Matemática **3**
1.1 Ambiente equation . . . 3
1.2 Ambiente align . . . 3
1.3 Ambientes Matemáticos . . . 3

2 Seção com Casos e Matrizes **3**
2.1 Função definida por partes . . . 3
2.2 Matriz e vetor . . . 4

3 Tikz **4**
3.1 Diagrama comutativo . . . 4
3.2 Borboleta . . . 4

4 Seção com Símbolos, Notação e Tabela **4**
4.1 Tabela de Simulação . . . 5
4.2 Tabela de Duas Colunas . . . 5
4.3 Tabela de Três Colunas . . . 5

5 Recursos do Template **5**
5.1 Código-Fonte com `minted` . . . 5
5.2 Código Lido de Arquivo . . . 7
5.3 Arquivo Longo . . . 7
5.4 Assembly, Terminal e Makefile . . . 9
5.5 Pseudocódigo com `algpseudocode` . . . 11
5.6 Unidades Físicas com `siunitx` . . . 11
5.7 Subfiguras com `subcaption` . . . 12
5.8 Referências Automáticas com `cleveref` . . . 12

1 Seção com Matemática

Texto com matemática inline: $e^{i\pi} + 1 = 0$, $\alpha + \beta = \gamma$ e $x^2 + y^2 = z^2$.

1.1 Ambiente equation

$$\int_0^{\infty} e^{-x^2}\, dx = \frac{\sqrt{\pi}}{2}. \tag{1}$$

1.2 Ambiente align

$$(a+b)^2 = a^2 + 2ab + b^2, \tag{2}$$
$$(a-b)^2 = a^2 - 2ab + b^2. \tag{3}$$

1.3 Ambientes Matemáticos

Os ambientes seguem o estilo visual do template e são compatíveis com `\cref`.

Definição 1.1 (Transformada Z) A Transformada Z bilateral de um sinal discreto $x[n]$ é definida como

$$X(z) = \sum_{n=-\infty}^{\infty} x[n]\, z^{-n}, \quad z \in \mathbb{C}.$$

Teorema 1.1 (Deslocamento no Tempo) *Se $X(z) = \mathcal{Z}\{x[n]\}$, então para qualquer inteiro k:*

$$\mathcal{Z}\{x[n-k]\} = z^{-k} X(z).$$

Demonstração Substituindo diretamente na Definição 1.1:

$$\sum_{n=-\infty}^{\infty} x[n-k]\, z^{-n} = z^{-k} \sum_{m=-\infty}^{\infty} x[m]\, z^{-m} = z^{-k} X(z). \qquad \square$$

Corolário 1.1 *O atraso de uma amostra corresponde a multiplicar por z^{-1} no domínio Z.*

Exemplo 1.1 Para $x[n] = \delta[n]$ (impulso unitário), tem-se $X(z) = 1$. Pelo Teorema 1.1, $\mathcal{Z}\{\delta[n-1]\} = z^{-1}$.

Observação A região de convergência (ROC) deve ser sempre especificada, pois sequências distintas podem ter a mesma expressão $X(z)$.

2 Seção com Casos e Matrizes

2.1 Função definida por partes

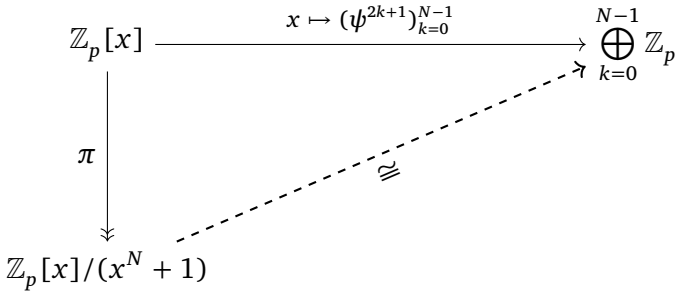
$$f(x) = \begin{cases} x^2, & x \geq 0, \\ -x, & x < 0. \end{cases} \tag{4}$$

2.2 Matriz e vetor

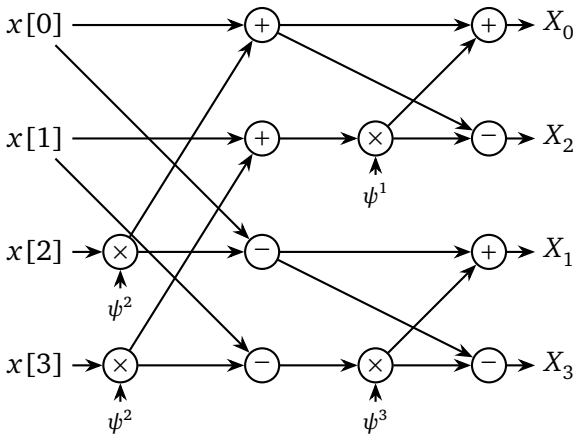
$$A = \begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 0 & 0 & 1 \end{pmatrix}, \qquad b = \begin{pmatrix} 1 \\ 0 \\ -1 \end{pmatrix}.$$

3 Tikz

3.1 Diagrama comutativo



3.2 Borboleta



4 Seção com Símbolos, Notação e Tabela

Seções

4.1 Tabela de Simulação 5
4.2 Tabela de Duas Colunas 5
4.3 Tabela de Três Colunas 5

Texto com símbolos: $\forall x \in \mathbb{R}$, $\exists y > 0$ tal que $|x| < y$ e $\sum_{k=0}^{n} k = \frac{n(n+1)}{2}$.

$$\lim_{n\to\infty} \left(1 + \frac{1}{n}\right)^n = e.$$

4.1 Tabela de Simulação

Tabela 1: Cenário 1 – Acerto das 16 jogadas (Simulação)

Caso	Operação	Sinais de entrada	Resultado esperado	Resultado observado
1	Resetar circuito	`reset_in = 1`	FSM no estado inicial	
2	Aguardar estabilidade	-	-	
3	Acionar iniciar	`iniciar_in = 1`	Aguardando jogada 1	Aguardando jogada 1
19	Acionar jogada 16	`chaves = 4'b0100`	Sinais **acertou** e **pronto** ativados	

4.2 Tabela de Duas Colunas

Tabela 2: Resumo de parâmetros do experimento

Parâmetro	Valor
Tamanho de palavra	16 bits
Clock	50 MHz
Nº de testes	16
Ambiente	Simulação (Icarus/GTKWave)

4.3 Tabela de Três Colunas

Tabela 3: Resumo do experimento

Item	Descrição	Observação
Clock	50 MHz	FPGA DE10-Lite
Entradas	4 bits (`chaves`)	Usuário
Saídas	`acertou`, `errou`	LEDs

5 Recursos do Template

5.1 Código-Fonte com `minted`

Para blocos com legenda e label, use `listing` (float) + `minted`:

Código 1: Contador de 4 bits em VHDL

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity contador is
  port (
    clk   : in  std_logic;
    reset : in  std_logic;
    q     : out std_logic_vector(3 downto 0)
  );
end entity;

architecture rtl of contador is
  signal count : unsigned(3 downto 0) := (others => '0');
begin
  process(clk)
  begin
    if rising_edge(clk) then
      if reset = '1' then
        count <= (others => '0');
      else
        count <= count + 1;
      end if;
    end if;
  end process;
  q <= std_logic_vector(count);
end architecture;
```

Código 2: Script de análise de simulação

```python
import numpy as np
import matplotlib.pyplot as plt

# CSV gravado pelo testbench: tempo, clk, q
data = np.loadtxt("sim.csv", delimiter=",", skiprows=1)
t, q = data[:, 0], data[:, 2]

plt.plot(t * 1e9, q, drawstyle="steps-post")
plt.xlabel("Tempo (ns)")
plt.ylabel("Saída q")
plt.savefig("forma_de_onda.pdf")
```

Para trechos inline use `\mintinline`, por exemplo: `if rising_edge(clk) then`.
Referências: Código 1 e Código 2.

5.2 Código Lido de Arquivo

Para não colar código no `.tex` (e deixá-lo desatualizado em relação ao programa que foi de fato medido), leia o trecho direto do arquivo. Marque o começo e o fim com o mesmo comentário `--tag--`, no próprio código:

```
// --conta--
int count_below_threshold(double array[], int n, double threshold) {
  ...
}
// --conta--
```

e inclua com `\inputtrecho[opções]{linguagem}{arquivo}{tag}`. As linhas das marcas não aparecem, a numeração é a do arquivo original e uma tag que não aparece duas vezes no arquivo gera erro de compilação:

Código 3: Função medida, lida de `codigos/count_below_threshold.c`

```
int count_below_threshold(double array[], int n, double threshold) {
  int count = 0;
  for (int i = 0; i < n; i++) {
    if (array[i] < threshold) {
      count++;
    }
  }
  return count;
}
```

Marcas aninhadas (uma tag dentro do trecho de outra) aparecem no trecho externo. Marca indentada funciona; para tirar a indentação do trecho, passe `autogobble` nas opções. Para linhas fixas:

```
\inputminted[firstline=10, lastline=18]{c}{arquivo}
```

5.3 Arquivo Longo

Um `minted` dentro de `listing` não quebra entre páginas: o float é uma caixa inteira, e o que passar de uma página estoura. Para um arquivo que pode ser longo, dispense o float e use o ambiente `codigolongo`, com a legenda como argumento e o `\label` logo depois. Ele não deixa a legenda sozinha no pé da página:

Código 4: Programa completo, lido de `codigos/count_below_threshold.c`

```c
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <time.h>


// --conta--
int count_below_threshold(double array[], int n, double threshold) {
  int count = 0;
  for (int i = 0; i < n; i++) {
    if (array[i] < threshold) {
      count++;
    }
  }
  return count;
}
// --conta--

void read_file_to_array(double **array, int *n, char filename[]) {
    FILE *f = fopen(filename, "r");

    if (!f) {
        fprintf(stderr, "Could not read file %s\n", filename);
        exit(1);
    }

    fscanf(f, "%d\n", n);
    *array = malloc(*n * sizeof(**array));
    for (int i = 0; i < *n; i++) {
        fscanf(f, "%lf\n", &(*array)[i]);
    }
    fclose(f);
}

static uint64_t now_ns(void) {
    struct timespec ts;

    if (clock_gettime(CLOCK_MONOTONIC, &ts) != 0) {
        perror("clock_gettime");
        return 0;
    }

    return (uint64_t)ts.tv_sec * 1000000000ULL +
           (uint64_t)ts.tv_nsec;
}


```

```c
int main(int argc, char const *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <0 or 1>\n", argv[0]);
        exit(1);
    }

    char filenames[2][100] = {
        "array0.dat",
        "array1.dat",
    };

    int mode = atoi(argv[1]);

    if (mode < 0 || mode > 1) {
        fprintf(stderr, "Mode should be 0 or 1\n");
        exit(1);
    }

    double *array = NULL;
    int n;
    read_file_to_array(&array, &n, filenames[mode]);

    uint64_t start = now_ns();
    int n_below_threshold = count_below_threshold(array, n, 0.3);
    uint64_t end = now_ns();

    uint64_t elapsed_ns = end - start;

    printf("Input file:                  %s\n", filenames[mode]);
    printf("Array length:                %d\n", n);
    printf("Below threshold:             %d\n", n_below_threshold);
    printf("Time (count_below_threshold): %lu ns\n", elapsed_ns);
    free(array);

    return 0;
}
```

Referência: Código 4.

5.4 Assembly, Terminal e Makefile

Para assembly, use `gascode` (saída de `gcc -S`) ou `objdumpcode` (saída de `objdump -d`). Para os outros formatos, use `makecode`, `bashcode` e `terminal` (sessão de shell, sem numeração). O Código 5 mostra que com `-O2` o `if` do Código 3 vira `seta`, sem desvio condicional.

O `gcc -S` separa mnemônico e operandos com tab, e o `minted` troca cada tab por um número fixo de espaços, sem respeitar as paradas de tabulação (`obeytabs` não resolve). Converta antes com `expand`: `gcc -O2 -S f.c -o - | expand > f.s`.

Código 5: Laço de `count_below_threshold` com `gcc -O2 -S`

```
.L5:
        xorl    %edx, %edx
        comisd  (%rdi), %xmm0
        seta    %dl
        addq    $8, %rdi
        addl    %edx, %eax
        cmpq    %rdi, %rcx
        jne     .L5
        ret
```

Código 6: O mesmo laço com `-O0`, via `objdump -d`: o `if` é o `jbe` em `0x41`

```
  3d:       comisd %xmm1,%xmm0
  41:       jbe    47 <count_below_threshold+0x47>
  43:       addl   $0x1,-0x8(%rbp)
  47:       addl   $0x1,-0x4(%rbp)
  4b:       mov    -0x4(%rbp),%eax
  4e:       cmp    -0x1c(%rbp),%eax
  51:       jl     20 <count_below_threshold+0x20>
```

Código 7: Regra de compilação e medição

```
aula1-count-below-threshold: aula1-count-below-threshold.c
        gcc -O0 -Wall -std=c99 $< -o $@
        perf stat ./$@ 1
```

Código 8: Contadores de hardware do vetor ordenado

```
$ perf stat -e cycles,instructions,branches,branch-misses
 ↪  ./aula1-count-below-threshold 1
Input file:                  array1.dat
Array length:                1000000
Below threshold:             299998
Time (count_below_threshold): 615251 ns

 Performance counter stats for './aula1-count-below-threshold 1':

       404,796,629      cpu_core/cycles/u
     1,892,450,688      cpu_core/instructions/u
       383,493,054      cpu_core/branches/u
         1,077,428      cpu_core/branch-misses/u

       0.082248693 seconds time elapsed
```

5.5 Pseudocódigo com `algpseudocode`

Opção `pseudo`. Cada linha começa com `\State` e cada bloco fecha com o seu `\EndIf`, `\EndFor`…; o `[1]` numera as linhas. Um `\label` logo depois de uma linha permite citá-la com `\cref`:

Algoritmo 1: Contagem de elementos abaixo do limiar

Require: vetor $A[0..n-1]$, limiar t
Ensure: quantidade de elementos com $A[i] < t$

```
1 function CountBelow(A, n, t)
2     c ← 0
3     for i ← 0 to n − 1 do
4         if A[i] < t then
5             c ← c + 1
6         end if
7     end for
8     return c
9 end function
```

O Algoritmo 1 descreve o laço do Código 3; o desvio condicional é a linha 4.

5.6 Unidades Físicas com `siunitx`

Use `\qty{valor}{unidade}` para quantidades e `\num{valor}` para números. A vírgula decimal e o separador de milhar seguem o padrão brasileiro automaticamente.

Exemplos de quantidades comuns em Engenharia Elétrica:

- Frequência de clock: 50 MHz
- Tensão de alimentação: 3,3 V
- Corrente de fuga: 1,5 µA
- Período de amostragem: 20 ns
- Resistência: 10 kΩ
- Capacitância: 100 pF

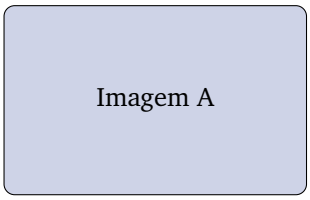
Número com separador de milhar: 1.234.567. Notação científica: $1{,}5 \times 10^{-9}$.

Tabela 4: Parâmetros de temporização

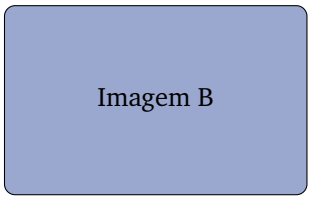
Parâmetro	Valor
Período de clock	20 ns
Setup time	1,5 ns
Hold time	0,25 ns
Tensão VCC	3,3 V
Corrente máxima	200 mA

5.7 Subfiguras com `subcaption`

Use o ambiente `subfigure` para colocar figuras lado a lado com legendas e rótulos independentes. Substitua os retângulos abaixo por `\includegraphics`.



(a) Primeira subfigura.



(b) Segunda subfigura.

Figura 1: Exemplo de subfiguras com `subcaption`.

5.8 Referências Automáticas com `cleveref`

Use `\cref{label}` no lugar de escrever `Figura~\ref{label}` na mão. O tipo da referência é inserido automaticamente em português:

- `\cref{fig:subfiguras}` → Figura 1
- `\cref{fig:sub-a}` → Figura 1a
- `\cref{fig:sub-a,fig:sub-b}` → Figuras 1a e 1b
- `\cref{tab:timing}` → Tabela 4
- `\cref{eq:integral}` → Equação (1)

`\Cref` (maiúsculo) é usado no início de frase: Figura 1 mostra um exemplo de subfiguras lado a lado.